This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Enhanced Vector Quantization for Embedded Machine Learning: A Post-Training Approach with Incremental Clustering

THOMMAS K. S. FLORES[1], MORSINALDO MEDEIROS[1], MARIANNE SILVA[2], DANIEL G. COSTA (Senior Member, IEEE) [3] and IVANOVITCH SILVA [1](Senior Member, IEEE)

[1]Federal University of Rio Grande do Norte/PPgEEC, Natal-RN, 59078–970, Brazil (e-mail: thommas.flores.101@ufrn.edu.br, morsinaldo.medeiros.075@ufrn.edu.br, ivanovitch.silva@ufrn.br)
[2]Federal University of Alagoas/SI, Penedo-AL, 57200-000, Brazil (e-mail: marianne.silva@penedo.ufal.br)
[3]SYSTEC-ARISE, Faculty of Engineering, University of Porto, Porto, 4200-465, Portugal (e-mail: danielgcosta@fe.up.pt)

Corresponding authors: Daniel G. Costa (e-mail: danielgcosta@fe.up.pt) and Ivanovitch Silva (e-mail: ivanovitch.silva@ufrn.br)

This paper received financial support from the Brazilian funding agency CNPq (National Council for Scientific and Technological Development), under the grant Process No. 405531/2022-2 and CAPES (Foundation for the Improvement of Higher Education Personnel) under the grant Process No. 88881.982076/2024-01

ABSTRACT TinyML enables the deployment of Machine Learning (ML) models on resource-constrained devices, addressing a growing need for efficient, low-power AI solutions. However, significant challenges remain due to strict memory, processing, and energy limitations. This study introduces a novel method to optimize Post-Training Quantization (PTQ), a widely used technique for reducing model size, by integrating Vector Quantization (VQ) with incremental clustering. While VQ is a technique that reduces model size by grouping similar parameters, incremental clustering, implemented via the AutoCloud K-Fixed algorithm, preserves accuracy during compression. This combined approach was validated on an automotive dataset predicting CO_2 emissions from vehicle sensor measurements such as mass air flow, intake pressure, temperature, and speed. The model was quantized and deployed on Macchina A0 hardware, demonstrating over 90% compression with negligible accuracy loss. Results show improved performance and deployment efficiency, showcasing the potential of this combined technique for real-world embedded applications.

INDEX TERMS Embedded systems, Post-training quantization, Vector quantization, Automotive sensors, Pollution

I. INTRODUCTION

The growing demand for Machine Learning (ML) and Deep Learning (DL) models in embedded systems has driven research into techniques that allow adapting complex models to resource-constrained platforms such as microcontrollers and mobile devices [1]. These systems are characterized by memory, processing capacity, and power consumption constraints, while being usually affordable and suitable for several operation scenarios. TinyML has emerged as a field that bridges the power of ML and DL with the limitations of embedded systems, unlocking new possibilities in complex areas such as Industry 4.0, smart cities, vehicular automation and healthcare [2]–[4]. In this work, the concepts of Machine Learning and Deep Learning are used interchangeably.

For TinyML applications to run effectively in real time, it is necessary to optimize the use of memory and processing resources [5]. These optimizations are essential to ensure that performance is not compromised while maintaining the accuracy of results, even on resource-constrained platforms [2]. This challenge is particularly relevant in areas where performance is a key factor [6], pushing us to investigate new optimization opportunities.

One of the major challenges to implementing ML models in embedded systems is their size and complexity [7]. Complex models, such as deep neural networks, often have millions of parameters that require significant storage and processing power [8]. Deploying these models on devices with limited memory and processing capabilities is particularly difficult, especially when memory usage must be minimized without compromising performance [9]. Furthermore, using large models can result in higher power consumption, which is problematic in battery-powered devices or in applications that require long periods of operation without recharging [10].

In this context, Post-Training Quantization (PTQ) emerges

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

as a promising technique to reduce the size of models and optimize their execution in embedded systems [11]. PTQ reduces the precision of model's parameters by representing them with lower-precision numbers (e.g., from 32 bits to 8 bits) [11], [12]. This results in a significant reduction in memory usage and computation time, in addition to improving performance in terms of execution speed [12], [13]. However, while quantization has proven effective in reducing model size, it can lead to loss of accuracy, which hampers the effectiveness of the model in critical applications such as predicting vehicle emissions or diagnosing faults in automotive systems [14].

The key challenge lies in balancing model size reduction and computational efficiency while preserving the accuracy of quantized models [15]. Moreover, as ML models are tuned for specific tasks, the distribution of parameters (weights and biases) can vary significantly, which directly impacts the performance of the model after quantization [16]. The strategy used to represent and store these parameters can determine both the efficiency of quantization and the accuracy of the final model [17]. Finding methods that optimize this representation, preserving the essential characteristics of the data while minimizing memory usage, is a central line of research [16], [17].

One efficient approach to reducing the size of ML models is Vector Quantization (VQ), a data compression technique [18]. VQ divides the model parameters (weights and biases) into groups of similar data and represents them by a single reference value, known as the centroid. This reduces the number of bits required to store the parameters [19]. This process involves grouping the data into clusters and using the indices of these clusters to represent the original values in a compact form [20]. VQ preserves the structure and essence of the data while improving memory and computation efficiency [18], [21]. This way, applying VQ to ML models can reduce the amount of memory required to store the parameters while maintaining a good approximation of the original values, which is important for operation on resource-constrained devices [21].

However, despite its expected effectiveness, VQ must be applied carefully, as the selection of cluster numbers and clustering methods directly impacts model accuracy and performance [20]. Achieving an optimal balance between compression and accuracy necessitates the use of advanced clustering and optimization techniques, which is inherently challenging [22]. Therefore, this article proposes a new approach to optimize the PTQ process by incorporating VQ techniques during the code generation phase and applying incremental clustering. This combined technique efficiently organizes datasets with similar characteristics, simplifying the data structure and significantly reducing the need for memory storage. Doing so, accuracy is mostly preserved while the required resources are reduced.

To perform the intended clustering, the AutoCloud K-Fixed method is considered, a clustering algorithm that dynamically adapts to data distributions [23]. With AutoCloud K-Fixed, the quantization process can accurately identify the representative centroids for each cluster, adjusting to the characteristics of the data extracted from the models [24].

To validate this approach, it will be considered an automotive dataset collected from vehicle sensors to predict CO_2 emissions based on measurements such as mass air flow (MAF), intake pressure, temperature, and speed. After building and training the Machine Learning model, the proposed solution will be quantized and deployed on a specific hardware, the Macchina A0, to evaluate the model's performance on a real embedded device. This test will allow us to verify the viability of applying the model to systems with limited hardware resources.

Furthermore, the model will be also compared with an alternative solution that uses the K-Means algorithm for clustering. This comparison aims to analyze the differences in performance between the two approaches, with a focus on model accuracy. Our study will also evaluate four research questions, examining the efficiency of parameter compression and providing clear insight into the trade-offs involved when applying different quantization techniques on resource-constrained devices.

The remainder of this article is organized as follows. Related works are discussed in Section II. The AutoCloud K-Fixed and K-Means algorithms are explained in Section III. The proposed approach is elaborated in Section IV. Section V outlines the case study employed for evaluating this approach. The principal findings are discussed in Section VI. Lastly, Section VII offers concluding remarks.

II. RELATED WORKS

Research on neural network optimization and quantization has significantly advanced computational efficiency while enabling the deployment of Machine Learning solutions on resource-constrained devices [25]. Recent studies have proposed several approaches to address specific challenges in this domain, including quantization methods that aim to reduce memory consumption and improve performance on hardware-constrained devices [21], [26]–[28].

In [21], authors investigated the performance of VQ compared to deep neural networks in clustering problems, a technique that is relevant to signal processing and information theory in electrical engineering. Their study proposed a K-means-based deep network architecture for VQ, which leveraged Deep Learning techniques to optimize convergence. By quantifying vectors over stacked neural layers, their architecture addressed the exponential complexity problem typically associated with VQ methods, with the existence of trainable parameters to improve efficiency. The results showed improvements over traditional clustering methods adapted for VQ. However, their approach still faces challenges in terms of scalability for very large datasets and the complexity of fine-tuning the network for diverse clustering tasks.

In a different work, an efficient quantization approach based on ''depthwise cluster quantization'' for deep neural networks was proposed in [29], aiming at model compres-

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

sion and memory overhead reduction. The technique divides the network weights into individual channels, groups them into clusters, and then performs quantization to force weight sharing. After quantization, the network is re-trained to adjust the quantized centroid values to improve accuracy. Experiments performed with ResNet-20 on datasets such as CIFAR showed a reduction in model size of up to 8 times (from 1.2MB to 152KB) while maintaining accuracy. This method is similar to the one we are proposing but with a specific focus on model quantization and compression to optimize memory and processing requirements in embedded devices.

From another perspective, the work in [26] proposed EQ-ViT, an acceleration framework for Vision Transformers (ViTs) targeting real-time ($< 1ms$) decisions. Their approach uses training-aware quantization to quantize weights and activations to 8 bits while maintaining accuracy. The EQ-ViT architecture leverages AMD Versal ACAP, combining FPGA and AIEs for acceleration, with up to 315x speedup and 62.2x energy efficiency gains. While EQ-ViT focuses on computer vision models and acceleration on specific platforms (ACAP), our work focuses on model quantization for prediction, with an emphasis on low-power embedded devices.

In contrast, [30] proposes the Low-Rank Adapters for Quantized Training (LoQT) method, which aims at efficient training of quantized models. The approach uses gradient-based factorization to initialize low-rank weight matrices, which are periodically combined with quantized full-rank weight matrices. The method is applicable for both pre-training and fine-tuning of models, allowing the training of models with up to 7 billion parameters on 24GB GPUs. Furthermore, it was demonstrated that it is possible to train a model with 13 billion parameters with per-layer gradient updates on the same hardware. Although the focus is on efficient training of large models on memory-constrained hardware, the research shares the same goal of optimizing quantized models for resource-constrained devices, such as in our case with embedded devices.

Some studies have applied quantization and optimization techniques in Machine Learning models specifically for vehicle data, aiming to improve computational efficiency in resource-constrained environments. In [27], authors presented a new quantization methodology for mixed-precision Convolutional Neural Networks (CNNs) optimized for edge computing platforms in automotive environments. The approach established a new Pareto frontier between model accuracy and memory consumption. In doing so, they demonstrated pre-trained quantized models capable of delivering high-level accuracy with less than 4.3 MB of weights and activations without changing the model architecture. Contributions include a tensor-sliced learned accuracy method and a targeted gradient modification. Furthermore, they introduced a multiphase learning schedule, which aims to address learning instabilities caused by updates to the learned quantizer and model parameters. Although promising, their methodology still faces challenges in terms of scalability to larger networks and adaptation to scenarios with different data types and hardware configurations.

In the work presented in [28], authors proposed a specialized regularization method for hybrid quantization of neural networks. Their goal is to optimize its implementation on ultra-low-power microcontrollers in automotive systems. The approach includes new regularization functions and a uniform hybrid quantization scheme for 2, 4, 8-bit configurations. In addition, the study introduced scheduling functions, focusing on timing challenges often neglected in quantization processes. The results showed a significant reduction in model size with minimal impact on accuracy, in addition to an effective minimization of energy and memory usage in microcontrollers. However, this approach still faces limitations regarding generalization across different types of network architectures and the need for further tuning to optimize quantization in scenarios with higher data variability.

Finally, recent research on model quantization for embedded devices has advanced significantly, showcasing various approaches aimed at enhancing computational efficiency and reducing memory usage without compromising accuracy. While much of this research targets resource-constrained devices, few studies specifically address implementation in automotive systems. Our work stands out by integrating advanced quantization techniques, including post-training quantization and vector quantization, with practical applications in embedded systems. This brings a new set of contributions not addressed by previous works.

III. FUNDAMENTALS OF THE EMPLOYED ALGORITHMS

In this section, we detail the K-Means and AutoCloud K-Fixed clustering algorithms (online technique), highlighting their characteristics and operation. Clustering is an unsupervised learning technique used to segment data into groups, or clusters, such that the elements within each group share high similarity, while the different clusters contain more distinct elements. These characteristics are relevant when constructing the proposed approach described in more detail in the following sections.

The comparison between K-Means and AutoCloud K-Fixed highlights the tradeoffs between static and adaptive clustering methods in the Post-Training Quantization process. K-Means, a traditional centroid-based algorithm, is widely recognized for its computational efficiency and simplicity, making it particularly suitable for static datasets with well-defined clusters. Its deterministic nature and ability to scale with data size have led to its success in several applications. [31] demonstrated that K-Means consistently performed well on data clustering tasks in the context of smart city metrics, highlighting its computational efficiency compared to algorithms such as Fuzzy C-Means and K-Medoids, especially in situations involving large volumes of data and stable features.

On the other hand, AutoCloud K-Fixed employs an evolutionary approach, allowing it to adapt to changes in data distributions over time, making it suitable for scenarios with low complexity and few classes. Unlike static methods, AutoCloud dynamically adjusts cluster structures as new data

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

streams enter, efficiently addressing challenges such as concept drift and data evolution. According to a recent study [32], AutoCloud has proven to be a useful alternative in simple data stream scenarios, standing out in situations where data complexity is reduced, even when compared to other evolutionary algorithms such as SOINN+ and CAEA.

By comparing these two algorithms, we intend to evaluate their performance in dealing with the weight clustering process and biases in the Post-Training Quantization process, exploring their strengths and limitations.

A. K-MEANS ALGORITHM

K-Means is a partitioning clustering algorithm that divides a dataset into a predefined number of clusters, denoted by K [33]. The algorithm operates iteratively, seeking to minimize the within-cluster variance. It starts by initializing K centroids, either randomly or through specific initialization methods such as K-Means++. Each data point is then assigned to the nearest centroid based on a distance metric, typically the Euclidean distance [34].

After assigning all points to clusters, the centroids are recalculated by taking the mean of all points assigned to each cluster. The process of reassigning points and updating centroids is repeated until convergence, which occurs when the cluster assignments no longer change or the centroids stabilize.

The objective function of K-Means is to minimize the sum of squared distances between each point and its corresponding cluster centroid, as described in Equation 1:

$$J = \sum_{i=1}^{K} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \tag{1}$$

where x_j is a data point in cluster C_i, and μ_i is the centroid of cluster C_i.

K-Means is efficient for large datasets due to its simplicity and low computational cost. However, it requires the number of clusters K to be specified in advance and may converge to local minima, depending on the initial centroids. Additionally, the algorithm assumes that clusters are spherical and of similar size, which may limit its performance on complex datasets.

B. AUTOCLOUD K-FIXED ALGORITHM

The AutoCloud K-Fixed algorithm is a variation of the AutoCloud algorithm that operates with a fixed number of data clouds during the clustering process [35]. Unlike the conventional AutoCloud, which can dynamically generate clouds indefinitely [36], K-Fixed limits the creation of new clouds to a predefined number.

The algorithm begins by taking as input the vector to be quantized ($X = \{x_1, x_2, \ldots, x_n\}$) and the desired number of clusters ($K_{\text{autocloud}} \in \mathbb{R}$, with $K_{\text{autocloud}} \geq 3$). The algorithm then iteratively analyzes each element x_k

If $k = 1$, a new data cloud is created with the first element x_1, initializing the mean (μ) and variance (σ^2) with the values of x_1, and its cardinality (s) is set to 1; or if $k = 2$, the second element x_2 is added to the first data cloud, and the mean and variance parameters are updated, increasing the cardinality to 2.

In addition, since it is a recursive algorithm, AutoCloud requires the storage of only three statistical characteristics for each data cloud c_i, which are:

- s_k^i: the number of samples belonging to data cloud i after reading k samples;
- μ_k^i: the mean of the samples belonging to data cloud i after reading k samples;
- $[\sigma^2]_k^i$: the variance of the samples belonging to data cloud i after reading k samples.

For $k \geq 3$, a dynamic process begins where each existing data cloud is examined to determine if the new element x_k should be added to it, or if a new data cloud should be created. This decision is based on metrics such as:

Typicity ($\tau_n(x)$): measures how representative a data point is within a given data cloud, as defined in Equation 2:

$$\tau_n(x) = 1 - \xi_n(x) \tag{2}$$

Eccentricity ($\xi_n(x)$): evaluates how much a data point deviates from the center of the data cloud, as defined in Equation 3:

$$\xi_n(x) = \frac{2 \cdot \pi_n(x)}{\sum_{i=1}^{n} \pi_n(x_i)}, \quad n \geq 2, \quad \sum_{i=1}^{n} \pi_n(x_i) > 0 \tag{3}$$

where $\pi_n(x)$ represents the sum of the distances from $x \in X$ to each of the other n samples.

These metrics assess whether the new element belongs to an existing cloud or if a new one should be created.

IV. PROPOSED APPROACH

This section presents the proposed approach aimed at enhancing the efficiency of the Post-Training Quantization (PTQ) process through the application of Vector Quantization (VQ) techniques and incremental clustering. The process is organized into three distinct processing phases, each serving a specific purpose in refining the PTQ process. Figure 1 illustrates the overall pipeline of the proposed approach, from data collection to actual deployment of the trained models.

As depicted in Figure 1, there are three key processing phases to be considered. Phase 1 involves defining the data and organizing it into test, validation, and training sets, which are then utilized to train and validate the machine learning model's structure. Then, Phase 2 focuses on extracting model parameters and transforming them into a vector of weights and biases through incremental clustering, forming a Codebook for quantization. Finally, Phase 3 entails converting the quantized model from a JSON format into C++ code, enabling integration into real-world systems. The final step involves embedding this C++ code into the target hardware, facilitating real-time estimation of the desired parameters. Figure 2 depicts the processing steps of each of these phases, which are further described in next subsections.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

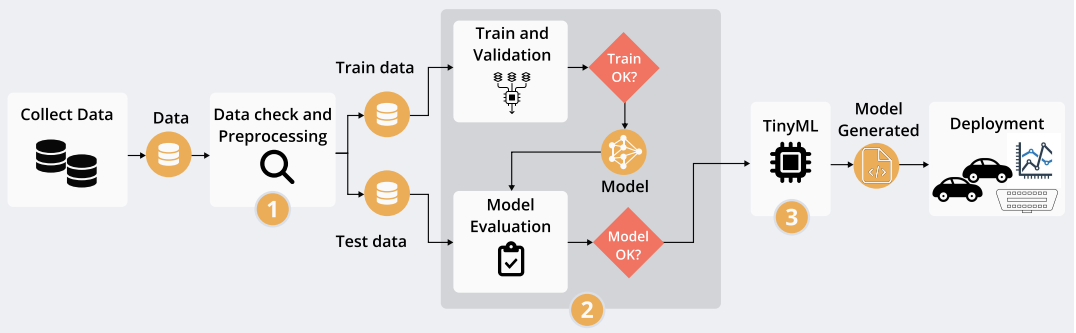


FIGURE 1. **The processing pipeline of the proposed approach.**

A. PHASE 1 - DATA CHECK AND PREPROCESSING

In this phase, the first step involves defining the dataset and organizing it into distinct subsets for training, validation, and testing. This preparation is important for ensuring that the machine learning model is trained effectively and evaluated properly.

Conceptually, there are two main functionalities expected from this phase, which are described as follows:

- Model Selection: The selection of a neural network architecture should be guided by the specific objectives and requirements of the task at hand. Commonly, architectures like Multilayer Perceptron Neural Networks may consist of an input layer, hidden layers with appropriate activation functions, and an output layer tailored to the desired output. The choice of architecture is important for effectively addressing the problem being tackled and should focus on achieving the best performance post-training.
- Training: During the training phase, a portion of the dataset is selected and shuffled to ensure a robust training process. The employed optimization algorithm facilitates effective learning, and the model undergoes multiple epochs to enhance its performance. Convergence is monitored through an appropriate loss function throughout this phase.

B. PHASE 2 - TRAIN AND VALIDATION

Phase 2 focuses on the task of extracting the model parameters. This phase is important for optimizing the model's representation and preparing it for deployment. Key characteristics of the model are extracted, including the number of layers, activation functions, and the values of weights and biases. This extraction process is essential for the subsequent steps. The extracted values are documented in a structured format, such as JSON, to facilitate further processing.

An important step in this phase is training evaluation, which systematically examines various machine learning models by altering their topologies and hyperparameters. Validation data is incorporated to mitigate overfitting. Several performance metrics can be utilized to assess the model's effectiveness, allowing for informed comparisons between different approaches.

After proper validation, the preparation for model compression starts in Phase 3.

C. PHASE 3 - TINYML

During this phase, vector quantization is performed. As described before, VQ is a technique used in data compression and representation, involving two main stages: encoding and decoding. In the encoding stage, input vectors are grouped into clusters, with each cluster represented by a single reference vector, known as a centroid [18]. This approach reduces data representation requirements while maintaining the integrity of the original information. The clustering method applied to the extracted model parameters aims to identify elements that best represent the distribution of weights and biases, enhancing data representation and compression. The decoding stage reconstructs the original data by using the centroid indices stored during encoding to retrieve the corresponding centroid values [20]. This ensures the model can approximate the original weights and biases effectively. Our approach focuses primarily on the encoding process, exploring its role in optimizing the representation of model parameters for efficient deployment.

For the clustering process in Phase 2, our approach focuses on the evolutionary method AutoCloud K-Fixed. The aim of this method is to identify the elements of the codebook that best represent the distribution of weights and biases of the Multilayer Perceptron (MLP) machine learning model.

After the extraction of the model parameters, the values of the weights and biases undergo a sequential processing. In this process, the values are extracted, resulting in two vectors: one for the weights and another for the biases. Next, we apply the clustering method to these vectors separately, which generates float-type centroids representing the grouped values.

The identified centroids are used to build a reference table, or Codebook. The next step involves calculating the Euclidean distance between each element of the weights and biases vectors and the corresponding elements of the Codebook. Each value in the vectors is then replaced by the integer index of the closest match found in the Codebook, allowing the

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

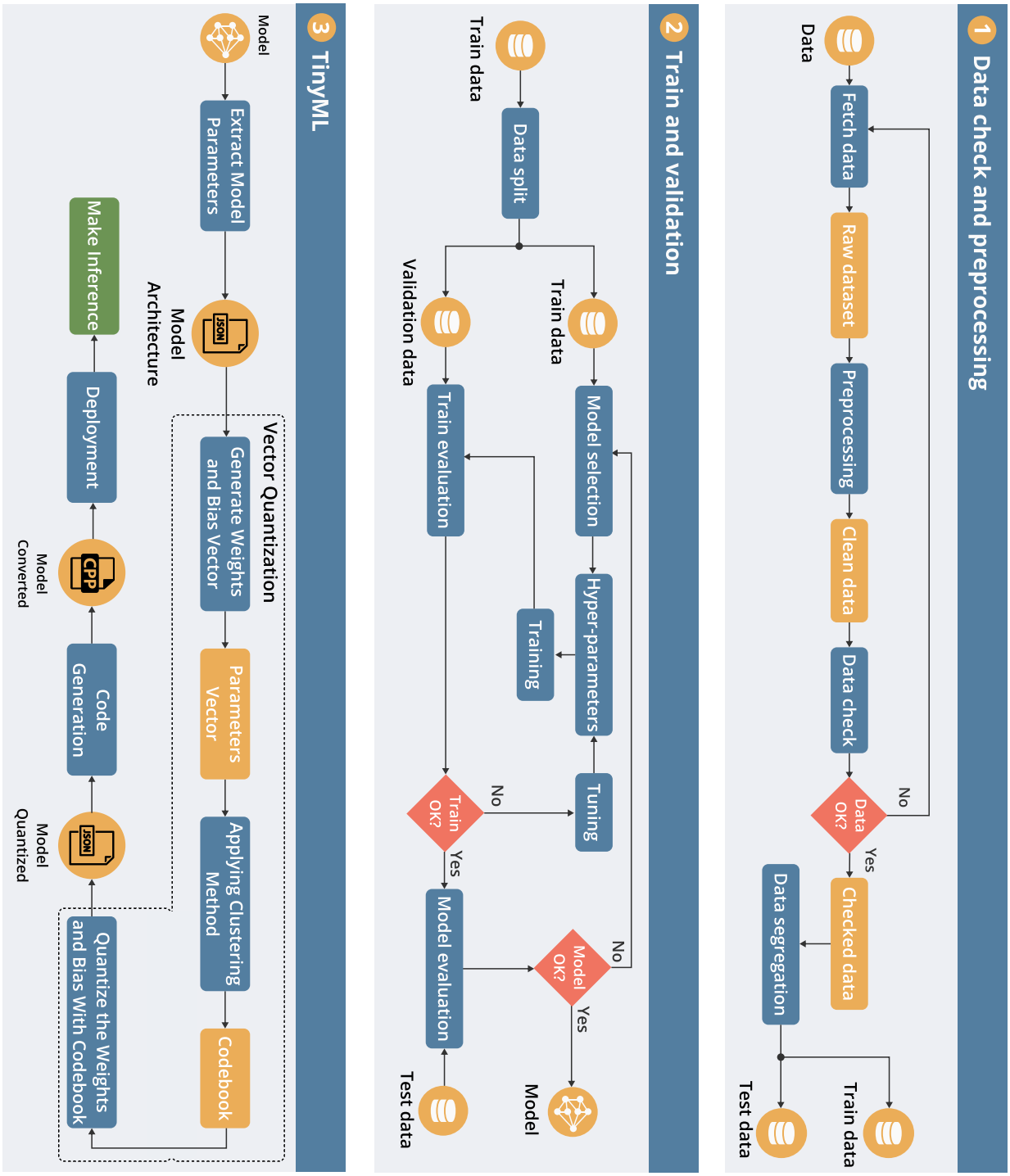


FIGURE 2. Main processing phases.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

model to store only these indices and significantly reducing memory requirements.

After this substitution, the weights and biases vectors are transformed back into matrices with the original dimensions of the model. This results in two new vectors containing the values of the centroids obtained during clustering. To perform inference, these matrices act as pointers that direct to the values stored in the Codebook. This entire process is depicted in Figure 3.

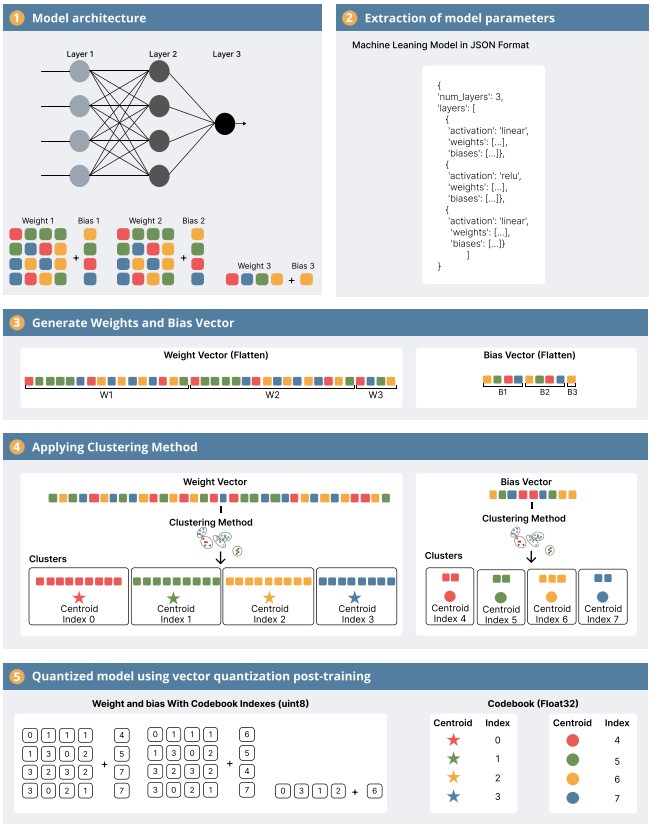


FIGURE 3. The schematic depicts the streamlined process for efficiently compressing model parameters, from initial extraction to final encoding.

Now, the focus shifts to the task of code generation, which transforms the trained machine learning model into an executable format suitable for deployment on resource-constrained devices. This step is for ensuring that the model can operate in real-world applications. The next steps are summarized as follows:

- Code Generation: The process of code generation involves transforming the trained ML model into a specific executable format suitable for deployment on resource-constrained devices. The model is translated from its structured format into the target programming language.
- Model Deployment: The integration of the quantized ML model is achieved through code generation. This step ensures that all model components, including weights, biases, activation functions, and mathematical operations, are accurately reproduced for the target hardware.
- Inference Evaluation: The final stage of the defined workflow evaluates the model's inference performance in the target hardware environment. Experimental setups are configured to generate the necessary input data, and the results are monitored and analyzed.

Although it is not a requirement, the entire development process is conducted in an open-source environment to ensure reproducibility and facilitate collaborative advancements.

V. CASE STUDY

In this section, we present a case study aimed at analyzing the proposed approach using an automotive dataset. The main research questions that will guide the intended analyses are:

1) **RQ1:** How do the size and complexity of the model impact its accuracy and efficiency in an automotive context?
2) **RQ2:** What is the impact of post-training quantization on the accuracy of automotive models?
3) **RQ3:** How are the distributions of weights and biases affected by quantization across different methods?
4) **RQ4:** What are the trade-offs between accuracy, memory efficiency, and energy consumption in the quantized models?

A. DATASET

The dataset used in this study comprises sensor data collected via the OBD-II system from three vehicles: Chevrolet Onix (model 2015), Ford Ka (model 2018), and Nissan Kicks (model 2019). The total size of the dataset is 10,230 samples and includes several measurements, such as intake pressure, intake temperature, speed, and MAF. The data was gathered using the Freematics OBD-II device and is provided by the Conect2AI research group [37] [38].

Given the importance of the MAF sensor in understanding engine performance and emissions, our analyses will leverage this data to predict CO_2 emissions. The MAF measurements are essential for calculating emissions and will be complemented by intake pressure, intake temperature, and speed as additional features [39].

Since CO_2 emissions are not directly measurable through the OBD-II system, we will estimate them using a derived Equation 4 that incorporates MAF and the other variables. This study did not employ a CO_2 sensor for real-time validation; instead, CO_2 emissions were estimated through mathematical models based on the vehicle's sensor data. Consequently, while the predicted values show good correlation with the expected emissions, a real-time comparison with sensor-measured CO_2 values was not conducted. This approach demonstrates the potential for estimating emissions using existing vehicle data, but future work could involve validating these estimates with direct CO_2 measurements for enhanced accuracy.

$$CO_2(g/s) = \frac{MAF(g/s)}{AFR \cdot Density_{Fuel}(g/l)} \cdot CO_2PL \qquad (4)$$

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

Finally, to enhance the performance of our regression model, we will preprocess the data by normalizing the input variables through min-max scaling, addressing any biases from differing value ranges.

B. PREPARATION

The preprocessed and verified data is divided into training and testing sets, with 70% allocated for training and 30% reserved for testing. Within the training set, 20% is further set aside for validation, while the remaining 80% is used for training the machine learning model. This structure is applied to the entire dataset.

In addition to data division, we selected a Multilayer Perceptron Neural Network as the model for this particular case study. The architecture consists of an input layer, followed by two hidden layers and an output layer. The input layer receives the data, while the first hidden layer applies a linear activation function and the second hidden layer employs the Rectified Linear Unit (ReLU) activation function, with each hidden layer containing twenty neurons. This design allows for effective learning and representation of the underlying patterns in the data.

For the training phase of the proposed approach, the dataset was randomly selected and subsequently shuffled. The Adam optimization algorithm was utilized, configured with a learning rate of 0.001, β_1 at 0.9, β_2 at 0.99, and ϵ at 10^{-7}. The model was trained using a batch size of 36 over 100 epochs.

C. INSTRUMENTATION

The model was trained using TensorFlow [40]. After the training phase, TensorFlow's internal functions were employed to extract key model characteristics, including the number of layers, activation functions, and the values of weights and biases. This extracted data was documented in a JSON format for easy reference.

For the coding process, Visual Studio Code (VS Code) [41] was utilized to convert the model characteristics into C++ code. This code was then compiled onto the microcontroller using the Arduino Integrated Development Environment (IDE) [42].

The Macchina A0 was selected for embedding the proposed model. Its availability and connectivity options, which include Bluetooth and Wi-Fi, made it a choice for this application. The technical specifications of the Macchina A0 are detailed in Table 1 [43].

TABLE 1. Macchina A0 specification.

Items	Specifications
License	Creative Commons 4.0 Int.
Main module	ESP32-WROVER-E
Microprocessors	Xtensa® 32-bit LX6
Integrated crystal	40 MHz
Integrated SPI flash	4 MB
Integrated PSRAM	8 MB
Clock frequency	up to 240 MHz
Performance	up to 600 DMIPS
Cores	2

D. OPERATION

In this study, we will compare two clustering methods: K-Means and AutoCloud K-Fixed. The parameters for both methods are outlined in Table 2.

TABLE 2. Parameters for the selected clustering methods.

Parameter	K-Means	K-Fixed
Number of Clusters (k)	3 to 18	3 to 18
Max Iterations	300	300
Initialization Method	K-Means++	-
Threshold Sensitivity (m)	-	0.1 to 1.4 (increments of 0.1)
Distance Metric	Euclidean	Euclidean

For the K-Means algorithm, we will evaluate various values of the number of clusters (k) ranging from 3 to 18. The maximum number of iterations will be set to 300 to ensure convergence, and the initialization method will utilize K-Means++ to improve convergence speed. The Euclidean distance metric will be employed to measure the distance between data points and cluster centroids.

Similarly, the AutoCloud K-Fixed method will also vary k from 3 to 18 for comparative analysis. Additionally, a threshold sensitivity parameter (m) will range from 0.1 to 1.4 in increments of 0.1, allowing us to evaluate the effect of sensitivity on clustering outcomes. The maximum number of iterations and distance metric will remain consistent with the K-Means approach, set at 300 and using Euclidean distance, respectively.

E. EVALUATION METRICS

In this study, several evaluation metrics were utilized to assess the performance of the machine learning models, each serving a specific purpose:

- Root Mean Square Error (RMSE): It was employed to evaluate the accuracy of the model predictions. RMSE measures the average magnitude of the errors between predicted and observed values, providing a clear indication of model performance. Lower RMSE values indicate better accuracy, making it essential for comparing different quantization methods.
- Compression Rate: The compression rate was calculated to assess the efficiency of the quantization process. This metric indicates the reduction in model size achieved through quantization, providing insights into the effectiveness of different methods. A higher compression rate signifies a more efficient approach, which is crucial for deployment on resource-constrained devices.
- Weights and Biases: The number of weights and biases in each model was recorded to analyze model complexity. This metric helps in understanding the potential for overfitting and the resource demands of the models. Reducing the number of weights and biases can lead to improved performance on embedded systems.
- Statistical Tests: Statistical tests, including Mann-Whitney and Kolmogorov-Smirnov, were conducted to compare the distributions of weights and biases across

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

different quantization techniques. These tests provide insights into the significance of differences observed between groups, with p-values indicating the robustness of the findings.

- Memory Usage (RAM and Flash): Memory usage metrics were collected to evaluate the resource requirements of each model when deployed on embedded systems. Understanding RAM and Flash memory usage is critical to ensure that models fit within the constraints of the target hardware.
- Inference Time: it was measured to determine how quickly each model can make predictions. Faster inference times are essential for real-time applications, making this a key evaluation criterion.
- Energy Consumption: it was assessed to evaluate the efficiency of each model in battery-powered or energy-sensitive applications. Lower energy consumption is desirable for optimizing performance in embedded environments.

By employing these metrics, we ensure a comprehensive evaluation of the model's predictive capabilities.

Finally, the scripts and libraries used are available on the GitHub repository [44], enabling researchers and practitioners to replicate this study and build upon this work.

VI. RESULTS AND ANALYSIS

This section presents the results of evaluating the scenarios defined in Section V, aiming to assess the efficiency of the vector quantization process in estimating greenhouse gas emissions (CO_2) in vehicles using OBD-II hardware. The results, obtained using only the three specified vehicle models, address the impact of quantization on accuracy, computational efficiency, and resource consumption, including memory and energy. Doing so, we want to answer the four stated research questions: how the size and complexity of the models influence their performance (RQ1), the effect of post-training quantization on accuracy (RQ2), the changes in weight and bias distributions due to quantization (RQ3), and the trade-offs between accuracy, memory efficiency, and energy consumption in the quantized models (RQ4), as specified in the previous section.

A. ANSWERING RQ1

To assess how model size and complexity affect accuracy and efficiency in an automotive context, three models (small, medium, and large) were trained ten times on the same dataset. This repeated training ensured the reliability of the results, allowing a thorough evaluation of their performance. The prediction errors for each model were as follows:

1) Small model: mean prediction error of -0.0194, with a 95% confidence interval between -0.0242 and -0.0146, and a margin of error of 0.0048.
2) Medium model: mean prediction error of -0.0009, with a 95% confidence interval from -0.0055 to 0.0037, and a margin of error of 0.0046.
3) Large model: mean prediction error of -0.0304, with a 95% confidence interval between -0.0350 and -0.0259, and a margin of error of 0.0046.

Although the large model has the highest number of parameters and file size, its mean prediction error is higher than the smaller models, indicating a trade-off between complexity and accuracy. Increasing model size does not necessarily improve performance and may lead to inefficiencies such as overfitting.

Table 3 provides a summary of the models' characteristics, including file size, number of parameters, and statistical metrics like RMSE, mean, standard deviation, skewness, and kurtosis, offering a more detailed view of their overall behavior.

The results indicate that all models have small error margins, suggesting consistency in their predictions. The skewness and kurtosis values reveal heavy tails in the error distributions, as is the case for the medium and large models, which points to a concentration of errors around the mean and the presence of potential outliers.

Therefore, it was observed that although larger models can improve accuracy, they come with higher memory and computational costs. Therefore, selecting the ideal model for automotive applications involves balancing prediction accuracy with resource efficiency.

B. ANSWERING RQ2

To evaluate the impact of post-training quantization on automotive models, ten instances of each model size (small, medium, and large (Section VI-A)) were tested using four quantization methods: K-Means, AutoCloud K-Fixed, Int8, and Float32. Accuracy was measured by RMSE, and efficiency was assessed based on the compression rate, calculated as the ratio between the original and quantized model sizes.

The quantization procedure involved transforming the weight and bias matrices into separate vectors, followed by clustering with the K-Means technique. The weights and biases were clustered separately, with different cluster combinations tested (e.g., weights in 3 clusters and biases in 3, 6, 9, 12, 15, and 18 clusters). This process was applied to both K-Means and AutoCloud K-Fixed, and the models were also quantized using Int8 and Float32 for comparison.

The results revealed distinct patterns across model sizes and quantization methods. For small models (Figure 4(a)), K-Means showed considerable variation in RMSE as the number of clusters increased. In contrast, the AutoCloud K-Fixed method demonstrated more consistent performance, with a consistently lower RMSE compared to K-Means. The Int8 and Float32 quantizations stood out for achieving the lowest RMSE, making them particularly effective for smaller models where both accuracy and model size reduction are important.

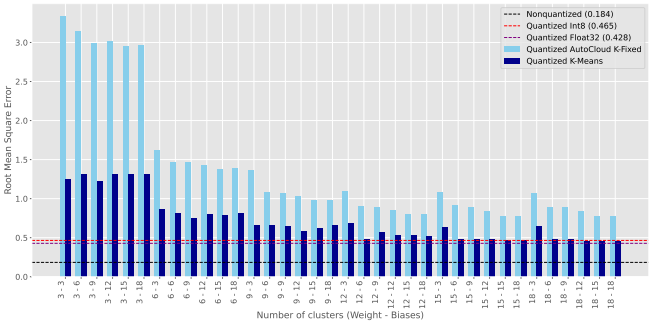
For medium models (Figure 4(b)), the RMSE for K-Means decreased as the number of clusters increased, although it still exhibited significant variation. AutoCloud K-Fixed maintained stable performance, often achieving a lower RMSE than K-Means. The Int8 and Float32 quantizations also performed well, maintaining low RMSE values, demonstrating

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

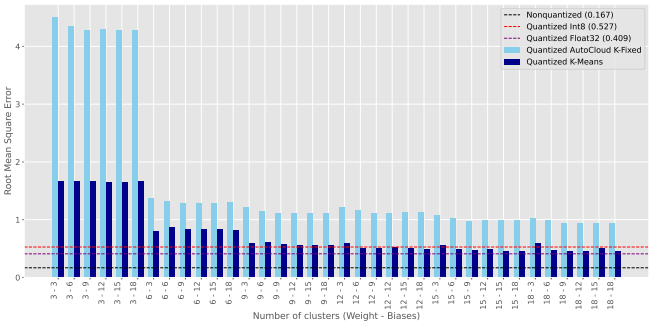
This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

TABLE 3. Model characteristics including size, parameters, and statistics.

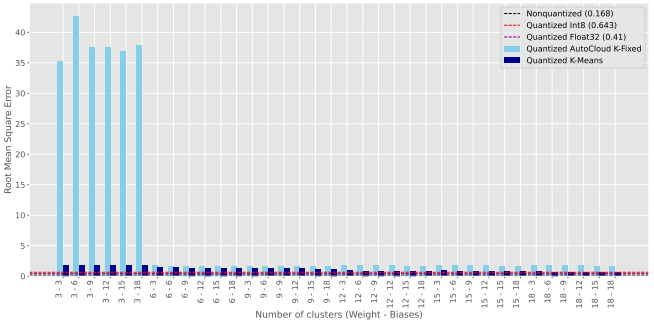
Model	File size (B)	Parameters	RMSE	Mean	Std	Skewness	Kurtosis
Small	26791	209	0.428	-0.019	$\pm$ 0.428	-2.44	15.65
Medium	70227	3009	0.409	0.0	$\pm$ 0.409	-2.71	18.26
Large	2175663	177281	0.410	-0.030	$\pm$ 0.409	-2.79	18.35



(a) Small model.



(b) Medium model.



(c) Large model.

FIGURE 4. Comparison of RMSE between quantized models.

their reliability for medium-sized models where a balance between accuracy and efficiency is required.

For large models (Figure 4(c)), K-Means maintained a low RMSE as the number of clusters increased, proving effective for larger and more complex models. However, AutoCloud K-Fixed produced a higher RMSE, and while Float32 achieved the lowest overall RMSE, it did so at the cost of a much lower compression ratio. Int8 quantization, on the other hand, provided a good compromise, offering accuracy and a reasonable compression ratio, making it an attractive option for larger models.

After analyzing Figure 4, which shows the RMSE of the quantized models, Table 4 complements this analysis by providing a detailed view of the compression rate and accuracy (measured by RMSE) of the different quantization methods for models of different sizes (small, medium and large).

TABLE 4. Comparison of different quantization methods applied to models of varying sizes.

Model	Quantization Method	Weights - Biases	RMSE	Compress Rate
Small (26791)	K-Means	18 - 15	0.45	9.46x
	AutoCloud K-Fixed	18 - 15	0.77	9.51x
	Float32	-	0.43	4.73x
	Int8	-	0.46	10.28x
Medium (70227)	K-Means	15 - 15	0.45	5.41x
	AutoCloud K-Fixed	18 - 9	0.94	5.3x
	Float32	-	0.4	1.05x
	Int8	-	0.53	3.92x
Large (2175663)	K-Means	18 - 15	0.59	3.46x
	AutoCloud K-Fixed	9 - 18	1.58	3.43x
	Float32	-	0.41	0.56x
	Int8	-	0.64	2.43x

Table 4 presents the number of clusters used in each method, the compression rates, and the RMSE values obtained after quantization, the results per model are:

1) Small models: K-Means achieved a good compression ratio of 9.46x, with an RMSE of 0.45, offering a good balance between size reduction and accuracy, although with variations in RMSE depending on the number of clusters. AutoCloud K-Fixed achieved a similar compression ratio (9.51x), but the RMSE was a little higher (0.77), indicating lower accuracy compared to K-Means. Float32, despite having the best RMSE (0.43), presented a lower compression ratio (4.73x), which indicates that, although accuracy is high, the reduction in model size is not as efficient. The Int8 method had the highest compression ratio (10.28x), with an RMSE of 0.46, making it a very efficient option for small models, offering a good compromise between compression and accuracy.
2) Medium models: For average models, K-Means had a compression ratio of $x5.41$ and an RMSE of 0.45, indicating good performance in terms of compression and accuracy. AutoCloud K-Fixed had a higher RMSE (0.94) and a compression of 5.3x, suggesting that although compression is similar to K-Means, accuracy is sacrificed. Float32 had a very good RMSE (0.40) but a very low compression ratio (1.05x), indicating that the reduction in model size was very limited. Int8 also had a reasonable RMSE (0.53) with a compression ratio of 3.92x, showing itself to be efficient in terms of

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

compression and accuracy, but with lower performance than K-Means in terms of RMSE.

3) Large Models: For large models, K-Means maintained a compression ratio of 3.46x and RMSE of 0.59, providing a good balance between compression and accuracy. AutoCloud K-Fixed, on the other hand, had a higher RMSE (1.58) and a compression of 3.43x, indicating that for larger models, accuracy was sacrificed more. Float32 achieved the best RMSE (0.41) but with a very low compression ratio (0.56x), suggesting that it is more effective in terms of accuracy but does not offer a significant reduction in model size. Int8 achieved a compression ratio of 2.43x and RMSE of 0.64, providing a good balance between accuracy and efficiency for large models.

Thus, K-Means stood out for its high compression rate in small and medium models but presented variations in RMSE depending on the number of clusters. AutoCloud K-Fixed was more stable in terms of RMSE, although with slightly lower precision. Int8 proved to be a good alternative, offering a good balance between compression and precision, with the highest compression for small models (10.28x) and a competitive RMSE (0.46). Float32 had the best precision but with a significantly lower compression rate in larger models.

C. ANSWERING RQ3

Quantization methods in machine learning models affect the distribution of model weights and biases in different ways. This study compared the distributions of parameters in the quantized model to the original (non-quantized) version, using descriptive statistics such as mean, standard deviation, minimum, and maximum values. This analysis aims to provide an understanding of how different quantization techniques alter the characteristics of the model's parameters.

As shown in Figures 5 and 6, the weight and bias distributions for the three model sizes (small, medium, and large) before and after quantization are illustrated. This highlights how the different quantization techniques affect the model parameters.

In the non-quantized model, the weights and biases exhibit a wide distribution, with a mean of 0.25 for weights and 0.1 for biases, and a standard deviation of 0.15 for both. Following quantization, the Int8 method compresses the range of values, producing a more compact distribution with a mean of 0.22 for weights and 0.08 for biases, and a reduced standard deviation of 0.12. While this results in a slight loss of precision, it greatly improves computational efficiency, making it ideal for low-power devices. In contrast, the Float32 method retains a distribution similar to the original model, with only slight reductions in range, maintaining accuracy at the cost of lower compression efficiency.

The AutoCloud K-Fixed method, which adjusts weights and biases to specific intervals, results in a concentration of values around fixed points, leading to reduced variance. With a mean of 0.23 for weights and 0.07 for biases, and a standard deviation of 0.11, AutoCloud K-Fixed strikes a good balance between accuracy and efficiency. K-Means quantization, which groups values into clusters, creates a more distinct distribution, with lower means (0.21 for weights and 0.06 for biases) and a higher standard deviation (0.13). While K-Means reduces variance it may introduce artifacts that affect accuracy, depending on the cluster initialization.

Statistical tests (Mann-Whitney and Kolmogorov-Smirnov) and RMSE values confirm that all quantization methods generally preserve accuracy well, with low RMSEs across different model sizes. K-Means and AutoCloud K-Fixed show more differences in the parameter distributions, but they still maintain effective accuracy. Float32 and Int8 also perform well, with low RMSE values, suggesting they are reliable methods for balancing precision and computational efficiency.

Overall, while quantization methods affect the distribution of model parameters, all tested techniques offer a balance between accuracy and compression efficiency, making them suitable for a variety of use cases. The choice of quantization method should be guided by the specific trade-offs between precision and computational efficiency for the intended application.

In addition, Tables 5 and 6 present the results of the statistical tests and the RMSE values for the weight and bias distributions in the quantized models, compared to the original model. The Mann-Whitney and Kolmogorov-Smirnov tests were used to evaluate the differences in the distributions of the parameters (weights and biases) between the quantized models and the original model, while the RMSE value measures the accuracy of the quantization to the original model.

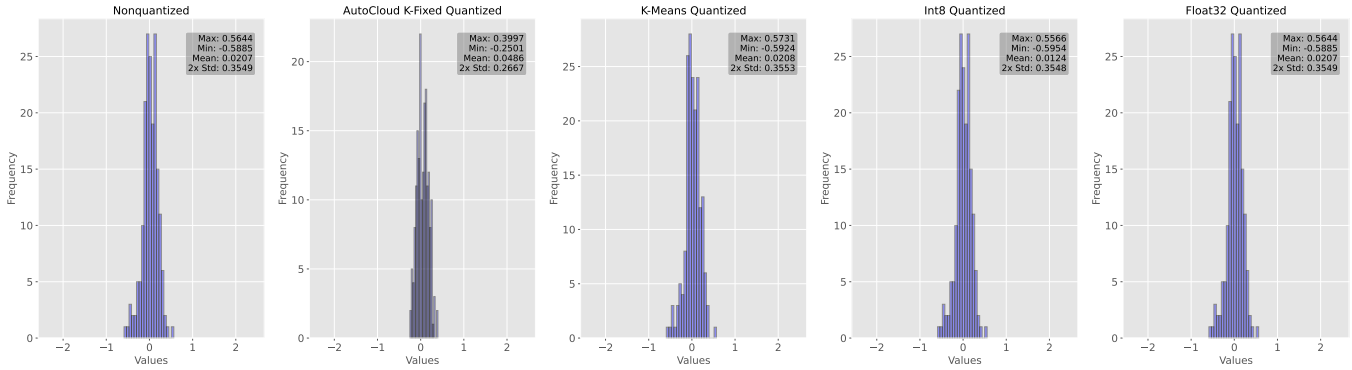
In the case of weights, Table 5 and demonstrates that, for the small model, the Int8 and Float32 methods better preserve the weight distribution, with relatively high p-values in the Mann-Whitney and Kolmogorov-Smirnov tests, indicating that the weight distributions of the quantized models are similar to those of the original model. On the other hand, the K-Means and AutoCloud K-Fixed methods show differences, with lower p-values, indicating that their weight distributions are more distinct from the original model. However, all methods present low RMSE values, suggesting that, although the weight distributions may have changed, the accuracy of the model was well preserved.

For the biases Table 6, the results follow a similar trend. For the small model, the K-Means and AutoCloud K-Fixed methods show differences in the bias distribution, but as with the weights, the RMSE remains low, which means that the accuracy of the biases was not severely compromised. The Int8 and Float32 methods also present good performances, with higher p-values in the Mann-Whitney and Kolmogorov-Smirnov tests, which indicates that their bias distributions are more similar to those of the original model, in addition to maintaining the low RMSE.

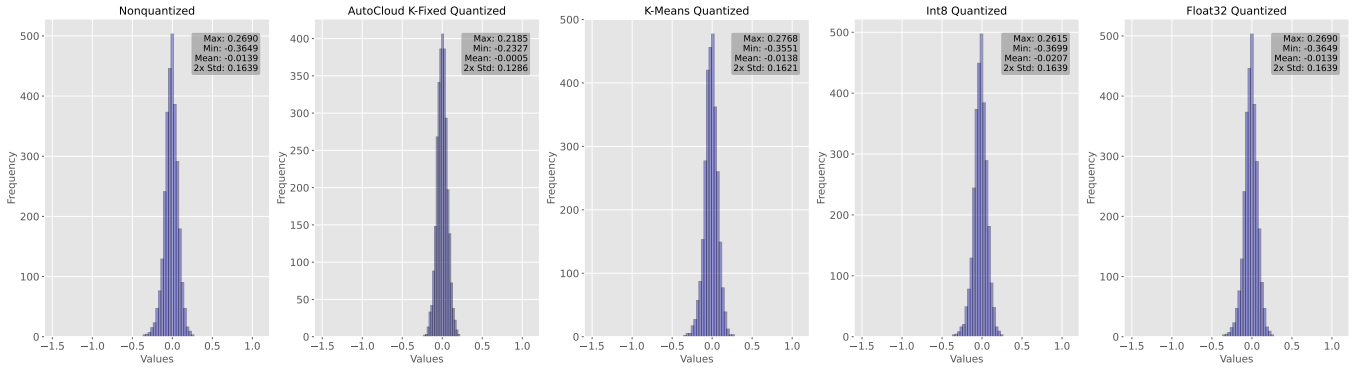
In medium and large models, the pattern repeats itself. The K-Means and AutoCloud K-Fixed methods continue to show differences in the weight and bias distributions, but keeping low RMSEs, which reinforces the idea that, despite

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

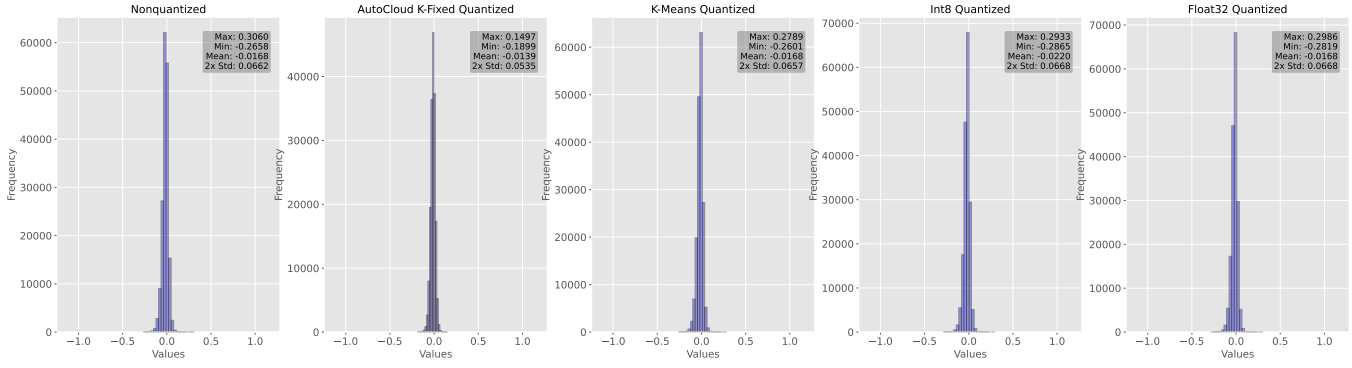
This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849



(a) Small model.



(b) Medium model.



(c) Large model.

FIGURE 5. Weight distributions in three MLPs (Small, Medium and Large) before and after the application of four quantization techniques.

the changes in the distributions, the accuracy of the model was not compromised. The Int8 and Float32 methods maintain the distributions closer to those of the original model with low RMSEs and preservation of accuracy.

D. ANSWERING RQ4

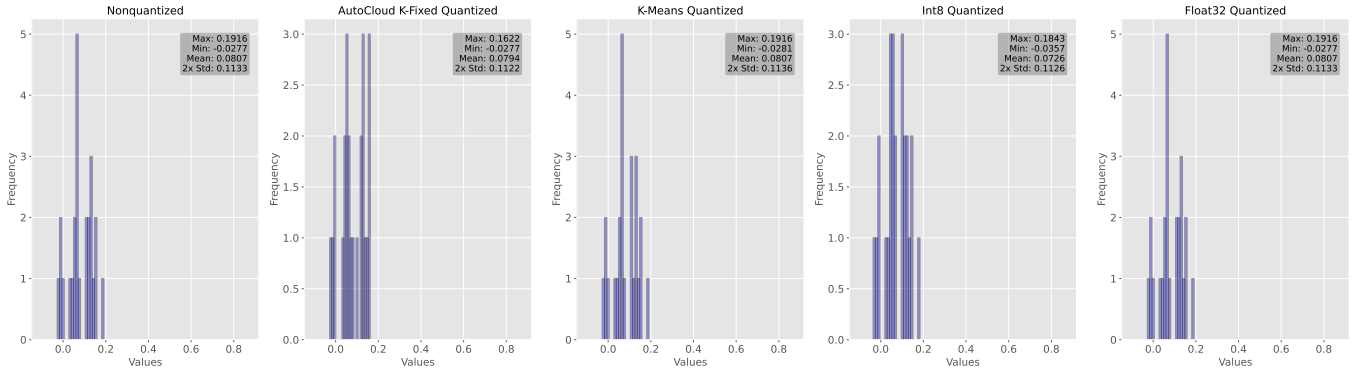
The quantized embedded models were evaluated based on performance metrics, including RMSE, RAM usage, Flash memory usage, inference time, and energy consumption. Thus, as in the other analyses, the quantization techniques (K-Means, AutoCloud K-Fixed, Float32, and Int8) were evaluated in models of three sizes: small, medium, and large, as shown in Table 7.

As can be seen in Table 7, for the small model, the AutoCloud K-Fixed method presented the highest RMSE (0.77), indicating a greater loss of accuracy compared to the other methods. Float32, on the other hand, maintained a lower RMSE (0.43), suggesting better preservation of accuracy. All methods used the same amount of RAM (16400 Bytes), and the Flash and energy consumption values were quite similar between the methods. The main difference was in the inference time, where Float32 had the shortest time (9 µs), while Int8 had a longer time (0.78 ms) as a result of the dequantization process, in which the inverse quantization function is used to make the inference.

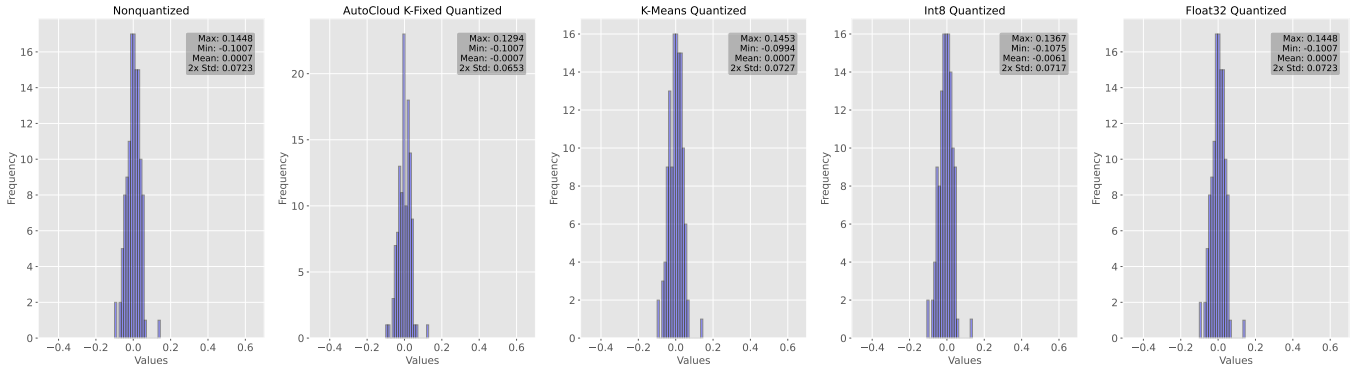
For the medium model, Float32 continued to show a good

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

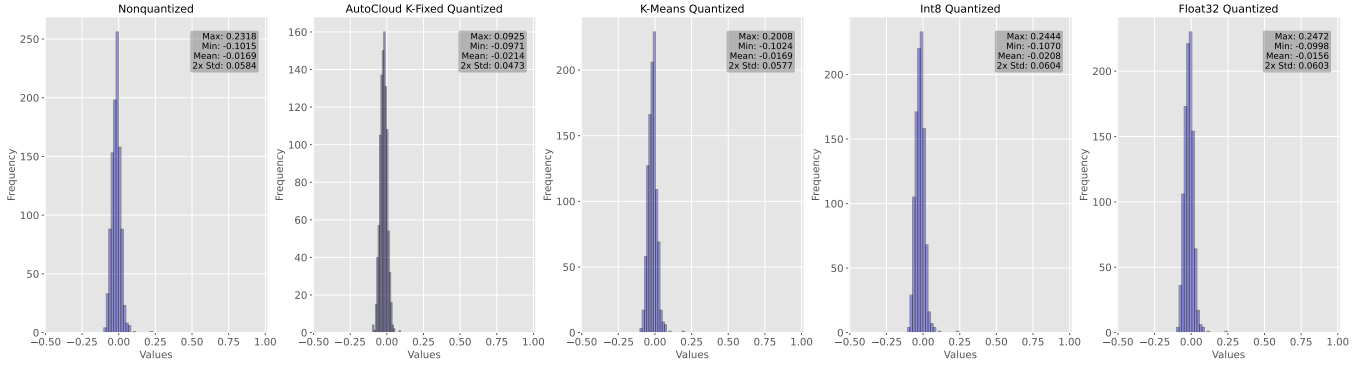
This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849



(a) Small model.



(b) Medium model.



(c) Large model.

FIGURE 6. **Bias distributions in three MLPs (Small, Medium, and Large) before and after the application of four quantization techniques.**

balance between accuracy and efficiency, with an RMSE of 0.4. AutoCloud K-Fixed performed worse, with an RMSE of 0.94, while K-Means had an RMSE of 0.45. While RAM usage remained constant for all methods (16,400 Bytes), Flash usage varied, with Float32 consuming the most memory (254,377 Bytes). Inference time was more variable, being 148 µs for K-Means and AutoCloud K-Fixed, while Int8 took 11.3 ms.

For large models, the differences between the quantization methods became clearer. K-Means had the highest RMSE (0.59), indicating greater loss of accuracy, while AutoCloud K-Fixed and Float32 maintained lower RMSE values (0.48 and 0.41, respectively). RAM usage was constant at 16400 Bytes, but Flash memory usage varied, with K-Means using 420609 Bytes and Float32 consuming 951697 Bytes. Inference time was highest for Int8 (0.94 s), while K-Means had the lowest time (0.28 s). Power consumption also varied, with Float32 consuming 340.8 mW, while Int8 required 376.8 mW.

These results reveal the trade-offs between accuracy, memory efficiency, and power consumption in quantization methods, answering the research question. While Float32 offers the best accuracy, it consumes the most Flash memory and power, which can be a limitation on resource-constrained embedded devices. K-Means and AutoCloud K-Fixed, on the other hand, have lower accuracy but offer better results in terms of computational efficiency and memory usage, making

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

TABLE 5. Statistical test results for weight in quantized machine learning models.

Model	Quantization	Weights - Biases	Mann-Whitney		Kolmogorov-Smirnov		RMSE
			Statistic	p-value	Statistic	p-value	
Small (26791)	K-Means	18 - 15	$1.69 \cdot 10^4$	0.995	0.022	1.0	0.013
	AutoCloud K-Fixed	18 - 15	$1.56 \cdot 10^4$	0.196	0.087	0.491	0.083
	Float32	-	$1.74 \cdot 10^4$	0.577	0.049	0.981	0.008
	Int8	-	$1.69 \cdot 10^4$	1.0	0.0	1.0	0.0
Medium (70227)	K-Means	15 - 15	$4.17 \cdot 10^6$	0.983	0.009	1.0	0.01
	AutoCloud K-Fixed	18 - 9	$3.78 \cdot 10^6$	0.0	0.093	0.0	0.038
	Float32	-	$4.37 \cdot 10^6$	0.001	0.044	0.007	0.007
	Int8	-	$4.17 \cdot 10^6$	1.0	0.0	1.0	0.0
Large (2175663)	K-Means	18 - 15	$1.55 \cdot 10^{10}$	0.883	0.002	0.862	0.004
	AutoCloud K-Fixed	9 - 18	$1.49 \cdot 10^{10}$	0.0	0.052	0.0	0.014
	Float32	-	$1.70 \cdot 10^{10}$	0.0	0.073	0.0	0.015
	Int8	-	$1.55 \cdot 10^{10}$	0.561	0.003	0.323	0.015

TABLE 6. Statistical test results for bias in quantized machine learning models.

Model	Quantization	Weights - Biases	Mann-Whitney		Kolmogorov-Smirnov		RMSE
			Statistic	p-value	Statistic	p-value	
Small (26791)	K-Means	18 - 15	$3.13 \cdot 10^2$	1.0	0.08	1.0	0.001
	AutoCloud K-Fixed	18 - 15	$3.15 \cdot 10^2$	0.969	0.12	0.996	0.01
	Float32	-	$3.51 \cdot 10^2$	0.461	0.16	0.915	0.008
	Int8	-	$3.13 \cdot 10^2$	1.0	0.0	1.0	0.0
Medium (70227)	K-Means	15 - 15	$7.33 \cdot 10^3$	0.982	0.041	1.0	0.002
	AutoCloud K-Fixed	18 - 9	$7.56 \cdot 10^3$	0.657	0.083	0.805	0.011
	Float32	-	$8.15 \cdot 10^3$	0.127	0.107	0.489	0.007
	Int8	-	$7.32 \cdot 10^3$	1.0	0.0	1.0	0.0
Large (2175663)	K-Means	18 - 15	$5.17 \cdot 10^5$	0.995	0.013	1.0	0.002
	AutoCloud K-Fixed	9 - 18	$5.65 \cdot 10^5$	0.0	0.105	0.0	0.012
	Float32	-	$5.58 \cdot 10^5$	0.002	0.079	0.004	0.011
	Int8	-	$5.05 \cdot 10^5$	0.34	0.054	0.102	0.01

TABLE 7. Comparison of the performance of embedded models.

Model	Quantization	Weights - Biases	RMSE	RAM (B)	Flash (B)	Inferece	Energy
Small (26791)	K-Means	18 - 15	0.45	16400	242757	14 μs	265.8 mW
	AutoCloud K-Fixed	18 - 15	0.77	16400	242753	14 μs	265.8 mW
	Float32	-	0.43	16400	243117	9 μs	264 mW
	Int8	-	0.46	16400	242753	0.78 ms	264 mW
Medium (70227)	K-Means	15 - 15	0.45	16400	245817	148 μs	265.4 mW
	AutoCloud K-Fixed	18 - 9	0.94	16400	245793	148 μs	264.8 mW
	Float32	-	0.4	16400	254377	86 μs	264 mW
	Int8	-	0.53	16400	245969	11.3 ms	264 mW
Large (2175663)	K-Means	18 - 15	0.59	16400	420609	0.28 s	346.6 mW
	AutoCloud K-Fixed	9 - 18	1.58	16400	420557	0.28 s	356.4 mW
	Float32	-	0.41	16400	951697	0.34 s	340.8 mW
	Int8	-	0.64	16400	420909	0.94 s	376.8 mW

them more suitable for devices with limited power and memory. Int8 has the lowest inference time and power consumption, making it ideal for applications that prioritize energy efficiency, although at a higher cost in terms of accuracy.

E. FURTHER DISCUSSIONS

In general, the results obtained during the experiments were compared with the methods presented in related works, focusing on metrics such as compression rate, accuracy, memory usage, energy consumption and inference time.

Regarding the compression rate, the proposed technique achieved up to 9.46x for small models, surpassing the average compression rates reported in related works. For instance, the study in [21] achieved a compression rate of up to 8x using a clustering-based approach for convolutional neural networks (CNNs). Similarly, [27] reported a compression rate of 4.3x for models optimized for automotive environments. The higher compression rate achieved by the proposed technique demonstrates its efficiency in reducing model size, making it particularly suitable for resource-constrained embedded devices.

In terms of precision, measured by RMSE, the proposed technique achieved competitive results. For small models, an RMSE of 0.45 (K-Means) and 0.43 (Float32) was observed,

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

comparable to the 0.41 reported in [27]. Notably, the proposed technique maintains low precision loss even with higher compression rates, highlighting its robustness.

For RAM usage, all methods used in the proposed technique maintained a constant value of 16.4 KB, aligned with the goal of optimizing for embedded devices. Conversely, works such as [27] and [28] did not provide detailed RAM usage but focused on reducing flash memory consumption, limiting direct comparisons.

The energy consumption of the proposed technique was competitive, with values optimized for the Macchina A0 hardware based on ESP32. In comparison, [26] reported energy efficiency gains of up to 62.2x using specialized hardware (FPGA). The proposed technique, however, focuses on low-cost, widely available hardware, offering practical advantages in cost-effective deployment.

In terms of inference time, the proposed technique demonstrated varied results depending on the quantization method. The Float32 method stood out with an inference time of 9 µs for small models, while the Int8 method required 0.78 ms due to the dequantization process. In contrast, [26] reported real-time inference (<1 ms) in specific contexts using hardware-accelerated systems.

In conclusion, this study significantly advances the state of the art by achieving superior compression rates, maintaining competitive accuracy, and ensuring resource efficiency. These contributions emphasize the practicality and versatility of the proposed approach, especially in contexts where cost and scalability are critical, thereby addressing limitations in previous works.

VII. CONCLUSIONS

This article proposed a new approach to optimize Post-Training Quantization (PTQ) by proposing the application of Vector Quantization (VQ) techniques during the code generation phase, along with the integration of incremental clustering methods. This approach aims to enhance the efficiency and effectiveness of quantization in embedded machine learning models, opening many possibilities to empower AI on the edge. The considered case study demonstrated that employing VQ techniques significantly reduced model sizes while preserving accuracy. By using incremental clustering, the proposed method not only optimized the quantization process but also improved the model's adaptability to varying data distributions, allowing for more efficient utilization of resources.

The achieved results indicated that the integration of VQ during code generation led to compression rates across different model sizes while preserving accuracy. For instance, the K-Means quantization method achieved an RMSE of 0.45 for a small model (26,791 parameters) with a compression rate of 9.46x. In comparison, the AutoCloud K-Fixed method demonstrated an RMSE of 0.77 with a similar compression rate of 9.51x, illustrating a favorable balance between model performance and resource efficiency.

The evaluation metrics further revealed that inference times were competitive and energy consumption was minimized, which is particularly advantageous for battery-operated devices. For a medium model (70,227 parameters), K-Means maintained an RMSE of 0.45 with a compression rate of 5.41x, while Float32 achieved an RMSE of 0.4 at a compression rate of only 1.05x, underscoring the importance of maintaining accuracy in applications where precision is critical. In large models (2,175.663 parameters), K-Means obtained an RMSE of 0.59 with a compression rate of 3.46x, indicating that effective quantization can still be achieved even as model sizes increase.

This dual focus on both model size reduction and operational efficiency positions the proposed method as a valuable tool for deploying machine learning models in resource-constrained environments. The application of quantization techniques is important in these settings, as it facilitates deployment on devices with limited memory and storage while maintaining computational efficiency.

For future works, it is suggested to explore additional quantization techniques beyond VQ and Incremental Clustering, such as binary quantization and energy level quantization, to further optimize the performance of the computed TinyML models. Additionally, investigating adaptive quantization strategies could allow models to adjust quantization levels based on input data characteristics, enhancing deployment flexibility.

ACKNOWLEDGMENT

The research presented in this paper received financial support from the Brazilian funding agency CNPq (National Council for Scientific and Technological Development), under the grant Process No. 405531/2022-2 and CAPES (Foundation for the Improvement of Higher Education Personnel) under the grant Process No. 88881.982076/2024-01.

REFERENCES

[1] M. Beltrán-Escobar, T. E. Alarcón, J. Y. Rumbo-Morales, S. López, G. Ortiz-Torres, and F. D. Sorcia-Vázquez, "A review on resource-constrained embedded vision systems-based tiny machine learning for robotic applications," *Algorithms*, vol. 17, no. 11, p. 476, 2024.

[2] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han, "Tiny machine learning: progress and futures [feature]," *IEEE Circuits and Systems Magazine*, vol. 23, no. 3, pp. 8–34, 2023.

[3] A. Elhanashi, P. Dini, S. Saponara, and Q. Zheng, "Advancements in tinyml: Applications, limitations, and impact on iot devices," *Electronics*, vol. 13, no. 17, p. 3562, 2024.

[4] A. Rocha, M. Monteiro, C. Mattos, M. Dias, J. Soares, R. Magalhães, and J. Macedo, "Edge ai for internet of medical things: A literature review," *Computers and Electrical Engineering*, vol. 116, p. 109202, 2024.

[5] R. Kallimani, K. Pai, P. Raghuwanshi, S. Iyer, and O. L. López, "Tinyml: Tools, applications, challenges, and future research directions," *Multimedia Tools and Applications*, vol. 83, no. 10, pp. 29015–29045, 2024.

[6] T. Flores, M. Silva, M. Azevedo, T. Medeiros, M. Medeiros, I. Silva, M. M. D. Santos, and D. G. Costa, "Tinyml for safe driving: The use of embedded machine learning for detecting driver distraction," in *2023 IEEE International Workshop on Metrology for Automotive (MetroAutomotive)*, pp. 62–66, IEEE, 2023.

[7] Z. Zhang and J. Li, "A review of artificial intelligence in embedded systems," *Micromachines*, vol. 14, no. 5, p. 897, 2023.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849

[8] N. N. Alajlan and D. M. Ibrahim, "Tinyml: Enabling of inference deep learning models on ultra-low-power iot edge devices for ai applications," *Micromachines*, vol. 13, no. 6, p. 851, 2022.

[9] E. Batzolis, E. Vrochidou, and G. A. Papakostas, "Machine learning in embedded systems: Limitations, solutions and future challenges," in *2023 IEEE 13th annual computing and communication workshop and conference (CCWC)*, pp. 0345–0350, IEEE, 2023.

[10] A. Fanariotis, T. Orphanoudakis, K. Kotrotsios, V. Fotopoulos, G. Keramidas, and P. Karkazis, "Power efficient machine learning models deployment on edge iot devices," *Sensors*, vol. 23, no. 3, p. 1595, 2023.

[11] J. Gutiérrez-Zaballa, K. Basterretxea, and J. Echanobe, "Evaluating single event upsets in deep neural networks for semantic segmentation: An embedded system perspective," *Journal of Systems Architecture*, vol. 154, p. 103242, 2024.

[12] F. Nakhle, "Shrinking the giants: Paving the way for tinyai," *Device*, vol. 2, no. 8, 2024.

[13] V. Finotti and B. Albertini, "Simulating quantized inference on convolutional neural networks," *Computers and Electrical Engineering*, vol. 95, p. 107446, 2021.

[14] R. Khoshkangini, M. Tajgardan, J. Lundström, M. Rabbani, and D. Tegnered, "A snapshot-stacked ensemble and optimization approach for vehicle breakdown prediction," *Sensors*, vol. 23, no. 12, p. 5621, 2023.

[15] E. Caldeira, P. C. Neto, M. Huber, N. Damer, and A. F. Sequeira, "Model compression techniques in biometrics applications: A survey," *Information Fusion*, vol. 114, p. 102657, 2025.

[16] L. Wei, Z. Ma, C. Yang, and Q. Yao, "Advances in the neural network quantization: A comprehensive review," *Applied Sciences*, vol. 14, no. 17, p. 7445, 2024.

[17] A. Kuzmin, M. Nagel, M. Van Baalen, A. Behboodi, and T. Blankevoort, "Pruning vs quantization: which is better?," *Advances in neural information processing systems*, vol. 36, 2024.

[18] S. Bruch, "Quantization," in *Foundations of Vector Retrieval*, pp. 127–142, Springer, 2024.

[19] W. Zhao, Q. Zou, R. Shah, and D. Liu, "Representation collapsing problems in vector quantization," in *Neurips Safe Generative AI Workshop 2024*.

[20] K. Nalini Sujantha Bel and I. S. Sam, "Image retrieval based on autoencoder and clustering with centroid update," *The Computer Journal*, p. bxae067, 2024.

[21] S.-K. Im and K.-H. Chan, "Vector quantization using k-means clustering neural network," *Electronics Letters*, vol. 59, no. 7, p. e12758, 2023.

[22] P. V. Dantas, W. Sabino da Silva Jr, L. C. Cordeiro, and C. B. Carvalho, "A comprehensive review of model compression techniques in machine learning," *Applied Intelligence*, pp. 1–41, 2024.

[23] M. Medeiros, T. Flores, M. Silva, and I. Silva, "A multi-layered methodology for driver behavior analysis using tinyml and edge computing," in *2024 IEEE International Conference on Evolving and Adaptive Intelligent Systems (EAIS)*, pp. 1–8, 2024.

[24] M. Silva, T. Medeiros, M. Azevedo, M. Medeiros, M. Themoteo, T. Gois, I. Silva, and D. G. Costa, "An adaptive tinyml unsupervised online learning algorithm for driver behavior analysis," in *2023 IEEE International Workshop on Metrology for Automotive (MetroAutomotive)*, pp. 199–204, IEEE, 2023.

[25] M. Li, Z. Huang, L. Chen, J. Ren, M. Jiang, F. Li, J. Fu, and C. Gao, "Contemporary advances in neural network quantization: A survey," in *2024 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–10, IEEE, 2024.

[26] P. Dong, J. Zhuang, Z. Yang, S. Ji, Y. Li, D. Xu, H. Huang, J. Hu, A. K. Jones, Y. Shi, Y. Wang, and P. Zhou, "Eq-vit: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on versal acap architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 3949–3960, 2024.

[27] C. J. Schaefer, S. Joshi, S. Li, and R. Blazquez, "Edge inference with fully differentiable quantized mixed precision neural networks," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 8460–8469, 2024.

[28] M. T. Lê, E. de Foras, and J. Arbel, "Regularization for hybrid n-bit weight quantization of neural networks on ultra-low power microcontrollers," in *International Conference on Artificial Neural Networks*, pp. 435–446, Springer, 2023.

[29] F. He, K. Ding, Z. Dong, J. Wang, M. Chen, and J. Li, "A novel quantization with channel separation and clustering for deep neural networks," in *2024 IEEE 4th International Conference on Electronic Technology, Communication and Information (ICETCI)*, pp. 632–637, 2024.

[30] S. B. Loeschcke, M. Toftrup, M. Kastoryano, S. Belongie, and V. Snæbjarnarson, "Loqt: Low-rank adapters for quantized pretraining," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[31] Ö. N. Kenger, Z. D. Kenger, E. Özceylan, and B. Mrugalska, "Clustering of cities based on their smart performances: A comparative approach of fuzzy c-means, k-means, and k-medoids," *IEEE Access*, vol. 11, pp. 134446–134459, 2023.

[32] T. Takebayashi, N. Masuyama, and Y. Nijima, "Adaptive resonance theory-based topological clustering with node deletion mechanism for evolving stream data," in *2023 International Conference on Machine Learning and Cybernetics (ICMLC)*, pp. 576–581, IEEE, 2023.

[33] Y. Chen, P. Tan, M. Li, H. Yin, and R. Tang, "K-means clustering method based on nearest-neighbor density matrix for customer electricity behavior analysis," *International Journal of Electrical Power & Energy Systems*, vol. 161, p. 110165, 2024.

[34] M. Ahmed, R. Seraj, and S. M. S. Islam, "The k-means algorithm: A comprehensive survey and performance evaluation," *Electronics*, vol. 9, no. 8, p. 1295, 2020.

[35] M. Silva, T. Medeiros, M. Azevedo, M. Medeiros, M. Themoteo, T. Gois, I. Silva, and D. G. Costa, "An adaptive tinyml unsupervised online learning algorithm for driver behavior analysis," in *2023 IEEE International Workshop on Metrology for Automotive (MetroAutomotive)*, pp. 199–204, IEEE, 2023.

[36] C. G. Bezerra, B. S. J. Costa, L. A. Guedes, and P. P. Angelov, "An evolving approach to data streams clustering based on typicality and eccentricity data analytics," *Information Sciences*, vol. 518, pp. 13–28, 2020.

[37] Freematics, "Freematics: Obd-ii telematics solutions," 2024. Accessed on: November 20, 2024.

[38] Conect2AI, "Conect2ai," 2024. Accessed on: November 20, 2024.

[39] P. Andrade, I. Silva, M. Silva, T. Flores, J. Cassiano, and D. G. Costa, "A tinyml soft-sensor approach for low-cost detection and monitoring of vehicular emissions," *Sensors*, vol. 22, no. 10, p. 3838, 2022.

[40] TensorFlow, "Tensorflow: Open source machine learning framework." https://www.tensorflow.org/, 2024. Accessed on: November 20, 2024.

[41] Microsoft, "Visual studio code: Code editing redefined." https://code.visualstudio.com/, 2024. Accessed on: November 20, 2024.

[42] Arduino, "Arduino software." https://www.arduino.cc/en/software, 2024. Accessed on: November 20, 2024.

[43] Macchina, "A0 under dash board - macchina." https://www.macchina.cc/catalog/a0-boards/a0-under-dash, 2024. Accessed on: November 20, 2024.

[44] Conect2AI, "Tinyml code generator - github repository." https://github.com/conect2ai/TinyML_code_generator, 2024. Accessed on: November 20, 2024.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3532849



THOMMAS K. S. FLORES earned an Electrical Engineering degree from the Federal University of Paraíba in 2018. In 2021, he obtained his Master's degree in Electrical Engineering from the same university, with a focus on adaptative control. Currently, as a doctoral student in Electrical and Computer Engineering at the Federal University of Rio Grande do Norte. His research focus on Embedded AI, MLOps, Internet of Intelligent Vehicles and Intelligent Controllers for industrial applications.



MORSINALDO A. MEDEIROS master's student in Electrical and Computer Engineering at the Federal University of Rio Grande do Norte (UFRN), specializing in Intelligent Information Processing. Holds a degree in Computer Engineering and a bachelor's degree in Science and Technology with an emphasis on Computing Technology, both from UFRN. Also holds a technical degree in Informatics from the Federal Institute of Rio Grande do Norte (IFRN). His research interests include Intelligent Vehicular Networks (IoV), Embedded Artificial Intelligence (TinyML), and Data Science.



MARIANNE SILVA is Professor at the Federal University of Alagoas (UFAL), Arapiraca Campus - Penedo Unit, teaching in the Information Systems program. Ph.D. in Electrical and Computer Engineering from the Federal University of Rio Grande do Norte (UFRN). Holds a Master's degree in Computer Science from the Federal University of Sergipe (UFS), and a specialization in Project Management from the Integrated Faculty of Patos (FIP). Graduated in Information Systems from Facisa. She is currently conducting postdoctoral research in the Graduate Program in Electrical and Computer Engineering (PPgEEC) at UFRN. Her primary research interests include Intelligent Vehicular Networks (IoV), Artificial Intelligence, Machine Learning, TinyML, and Data Science.



DANIEL G. COSTA (SENIOR MEMBER, IEEE) is an Assistant Professor of the Department of Electrical and Computer Engineering (DEEC), Faculty of Engineering (FEUP), University of Porto. He is an author or co-author of more than 100 international scientific papers in prestigious journals and conferences, covering research topics in the areas of Smart Cities, Internet of Things, Sensor Networks, Embedded Systems and Multimedia Sensing.



IVANOVITCH SILVA (SENIOR MEMBER, IEEE) earned his B.S. (2006), M.Sc. (2008), and Ph.D. (2013) degrees in Electrical and Computer Engineering from the Federal University of Rio Grande do Norte (UFRN), Brazil, with a co-participation from the University of Porto Portugal. Currently, he is an Associate Professor at UFRN's Department of Computer Engineering and Automation (DCA), and in 2021, he served as a Visiting Professor at the Università degli Studi di Brescia, Italy. His research interests focus on Embedded AI, MLOps, Internet of Intelligent Vehicles and Sustainable Cities.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/